

FFgif — Test Report

Hand-rolled mocks replaced with `go.uber.org/mock`; suite written to the interface contract rather than to current behaviour. **29 tests fail, and every failure names a real defect.** No implementation code was patched.

Repo `github.com/labib0x9/ffgif` Toolchain Go 1.27.1 (project-local) Run `go test -race ./...` Date 9 September 2026

305 PASSING incl. subtests	29 FAILING 54 with subtests	14 SKIPPED need live infra	0 DATA RACES under -race
---	--	---	---

1 Environment

No Go toolchain existed on this machine. Go 1.27.1 was installed **project-locally** in `.devtools/` (SHA-256 verified against `go.dev`), with `GOPATH`, `GOCACHE`, `GOMODCACHE` and `GOTMPDIR` all inside that directory — deleting the project deletes the toolchain. Use `.devtools/go.cmd` or `source .devtools/goenv.sh` rather than a global `go`. Added to `.gitignore`.

Docker Desktop could not be installed

It requires administrator elevation, and this session is not elevated — the UAC prompt has no one to answer it. It also cannot be made project-local. The 14 integration tests are written and gate cleanly on service reachability; they run under `docker compose up -d`.

Note: an **unrelated Postgres is already listening on 127.0.0.1:5432** on this machine. A plain TCP probe therefore passes while the credentials fail, and `NewPostgresConn` panics rather than returning an error. The gate authenticates and checks for the `gifs` table, recovering that panic into a skip, so a developer's own Postgres never looks like a test failure.

2 Mock migration

`//go:generate mockgen` directives were added to every interface file; `go generate ./...` produces **18 mock files** covering all 17 interfaces requested plus the four application `Service` interfaces needed for handler tests. Regeneration is idempotent and the tree builds clean.

Every hand-rolled mock is gone — verified zero `type mock*` declarations remaining across `internal/` and `tests/`. The duplicated fake `AuthRepository` that previously existed in four separate test files is now one generated implementation.

One production-code addition, the one the brief asked for: `ErrCacheMiss` in `internal/port/cache/errors.go`. Nothing returns it yet — that absence is precisely what finding 14 asserts. No other implementation file was modified.

3 Findings — 29 failing tests

Grouped by area, most severe first. Every row is a test that fails on the current code, with a message that names the defect. File references are the source of the defect, not the test.

#	TEST	DEFECT LOCATION	WHAT IS WRONG
AUTHORIZATION & ACCESS CONTROL			
1	TestAuth_BlocklistFails ClosedWhenRedisIsDown	transport/http/middleware/auth. go	Blocklist fails open. <code>if _, err := cache.Get(...); err == nil</code> treats only a nil error as "blocked". A connection refusal, timeout or <code>NOAUTH</code> all read as "not blocklisted", so a Redis outage silently revalidates every token users logged out with, for the whole outage.

#	TEST	DEFECT LOCATION	WHAT IS WRONG
2	TestStream_NonOwnerCannotMintAStreamUrl	app/media/stream.go	IDOR. <code>Stream(ctx, userId, key)</code> accepts <code>userId</code> and never reads it — it goes straight to <code>storage.GetStreamURL(key)</code> . Any authenticated user can mint a presigned stream URL for any object key. Every sibling method checks <code>GetOwner</code> first.
3	TestSave_NonOwnerCannotSaveSomeoneElsesGif	app/media/save.go	IDOR. The body is <code>// To-do, validate user</code> followed by an unconditional <code>SaveRecent(key)</code> . Any user can pin any other user's GIF into their recents by key.
4	TestForgotPassword_ResponseIsIdenticalForKnownAndUnknownAddresses	handlers/auth/forgot_password.go	Account enumeration. A registered address returns <code>202 {"msg": "check mail"} ;</code> an unregistered one returns <code>404 AUTH_USER_NOT_FOUND</code> . The difference lets anyone test which emails hold accounts.

CREDENTIAL HANDLING

5	TestForgotPassword_StoresHashOfEmailedToken	app/auth/forgot_password.go	Raw reset token at rest. <code>resetToken, _ := token.GenerateToken()</code> keeps the raw token and discards the hash, then writes the raw value into <code>reseter.token_hash</code> . Anyone with read access to the table — a backup, a replica, a log — can reset any account. The signup flow does this correctly, which is the contrast the test draws.
6	TestCreateByToken_StoresHashNotRawBearerToken	app/share/create_by_token.go	Raw bearer token at rest. Same pattern: <code>share_tokens.token</code> holds the working share credential in plaintext.
7	TestForgotPassword_DoesNotReuseAlreadyUsedToken	app/auth/forgot_password.go	The existing row is reused without inspecting <code>Reseter.Used</code> , so a token that has already been redeemed is emailed out again and stays live for its full TTL.
8	TestResetPasswordPost_MismatchedConfirmationIsRejected	app/auth/reset_password.go	<code>confirmPass</code> is accepted as a parameter and never read. The only mismatch check is the HTTP handler's <code>eqfield=Password</code> tag, so any non-HTTP caller resets the password with no confirmation at all.
9	TestResetPasswordGet_DoesNotEchoStoredCredential	app/auth/reset_password.go	The validity-probe endpoint returns the stored <code>reseter.Token</code> verbatim. Combined with finding 5, it hands back a directly usable credential.
10	TestChangePassword_RefusesToReuseTheCurrentPassword	app/user/change_password.go	Setting the password to its current value is accepted and rewrites the row with a fresh bcrypt hash. The user is told the rotation succeeded while the credential is unchanged.

QUOTA — TRACKED BUT NEVER ENFORCED

11	TestUpload_IsRejectedWhenTheUserIsOverQuota	app/media/upload.go	<code>QuotaRepository</code> is injected into the media service and <code>Quota</code> carries <code>UsedBytes</code> , <code>TotalBytes</code> , <code>GifCount</code> and <code>gif_limit</code> — but no code path in <code>internal/app/media</code> reads any of it. A user at 100% is still issued presigned upload URLs.
12	TestConvert_IsRejectedWhenTheUserIsAtTheirGifLimit	app/media/convert.go	Same gap on the conversion path: a user at their <code>gif_limit</code> can enqueue unlimited jobs, each producing another GIF.
13	TestConvert_ConcurrentJobsCannotDoubleSpendTheLastQuotaSlot	app/media/convert.go	20 concurrent conversions against a single remaining slot: all 20 are enqueued. With no check there is nothing to race against; once one exists this test guards it from a TOCTOU regression.

#	TEST	DEFECT LOCATION	WHAT IS WRONG
ERROR SEMANTICS & STATE			
14	TestStatus_CacheMissIsNotAnInternalError	app/media/status.go	Every <code>cache.Get</code> error is returned as-is, and go-redis signals a missing key with <code>redis.Nil</code> — so an ordinary cache miss becomes a 500 . The <code>status == ""</code> → <code>"failed"</code> branch below it is dead code that can never be reached. This is the inverse of what the brief described.
15	TestConvert_PublishFailureDoesNotLeaveAPhantomQueuedJob	app/media/convert.go	Two cache entries are written before publishing. If the publish fails they are left reading <code>"queued"</code> for a job no worker will ever see, and only clear after the 5-minute TTL. The client polls a job that will never run.
16	TestUpload_RejectsFilenameWithNoExtension	app/media/upload.go	The object key is derived with <code>filepath.Ext</code> and never validated. <code>"", "noextension"</code> and <code>"trailingdot."</code> all yield upload slots. <code>media.ErrInvalidExt</code> exists for this and is unused on the path.
17	TestCreate_RejectsNonFutureExpiry	app/share/create.go	<code>expiresAt</code> is never inspected. A share expiring in the past, exactly now, or at the zero value is written to the DB, where <code>expires_at > now()</code> filters it out immediately. The caller is told it succeeded; the recipient can never open it.
REQUEST VALIDATION			
18	TestConvert_RejectsTrimWindowWhereEndIsNotAfterStart	handlers/media/convert.go	<code>convertRequ</code> validates <code>Start</code> and <code>End</code> independently (<code>gte=0</code> , <code>gt=0</code>) and never relates them. <code>start=5</code> , <code>end=2</code> returns 202 and hands ffmpeg a –3-second window; <code>start=5</code> , <code>end=5</code> gives a zero-length one.
19	TestCreate_ValidatesTheRecipientAndExpiry	handlers/share/create.go	<code>reqCreate</code> carries no <code>validate</code> tags and the handler never calls <code>validate.Struct</code> — unlike every other write handler. An empty recipient, a non-email string, and a zero-value <code>expire_at</code> all return 201 .
20	TestConvert_UnauthenticatedRequestIsUnauthorizedNotServerError	handlers/media/convert.go	The handler validates the body before reading the user id from the context and answers 500 when it is absent. It should be 401 — and should not first tell the caller whether their payload parsed.
21	TestSignup_LocationHeaderPointsAtTheCreatedUser	handlers/auth/signup.go:50	<code>w.Header().Set("Location", "/users/"+res.Id)</code> — the Go expression was written as a string literal. Every signup returns <code>Location: /users/res.Id</code> , pointing at nothing.
WORKERS — FIRST FAILURE IS FATAL			
22	TestVideoWorker_TransientFailureIsRetriedNotDeadLettered	worker/converter_worker.go	Every processing failure answers <code>Nack(false, false)</code> — straight to the DLQ on the first attempt. The <code>retryCount()</code> helper that reads the <code>x-death</code> header is commented out and the <code>maxRetries</code> field (set to 2) is never read. An ffmpeg OOM or MinIO blip permanently loses the conversion.
23	TestEmailWorker_SmtpTimeoutIsRetriedNotDeadLettered	worker/email_worker.go	Same pattern. An SMTP timeout — the commonest transient fault in an email pipeline — permanently drops the verification mail, so the user can never confirm their account.
24	TestSaveWorker_TransientStorageFailureIsRetried	worker/save_metadata_worker.go	Same pattern. A MinIO <code>RequestTimeout</code> loses the upload's metadata for good — the video sits in the bucket but never appears in the user's library.

#	TEST	DEFECT LOCATION	WHAT IS WRONG
25	TestEmailWorker_HandlesShareNotifications	worker/email_worker.go	Share emails are never sent. <code>share/create_by_token.go</code> publishes a job named "share", but the worker's switch has no case for it — only <code>signup</code> , <code>forgot-password</code> , <code>resend-verify</code> and <code>reset-password</code> . Every share notification hits <code>default:</code> and is dead-lettered. <code>mailer.SendShareNotification</code> is declared and never called anywhere in the codebase.

ROBUSTNESS & ARCHITECTURE

26	TestRateLimiter_MalformedScriptReplyDoesNotPanic	transport/http/middleware/rate_limiter.go	Panics, four ways. <code>setLimit</code> does an unchecked <code>res.([]interface{})</code> plus three unchecked <code>.(int64)</code> assertions and an unguarded index. A non-slice reply, a short slice, a wrong element type or <code>nil</code> each panic inside the middleware chain instead of returning 500.
27	TestRateLimiter_SubSecondWaitIsNotAdvertisedAsZero	transport/http/middleware/rate_limiter.go	<code>retryAfterSecs := res.wait_ms / 1000</code> is integer division, so any wait under a second is advertised as <code>Retry-After: 0</code> . A well-behaved client retries immediately and spins — the header causes the hammering it exists to prevent.
28	TestPortPackagesDoNotLeakDriverTypes	port/queue/repository.go	Driver SDK in a port contract. <code>ConsumeEmail(...)</code> (<code><-chan amqp.Delivery, error</code>) puts a broker-specific struct — carrying an <code>Acknowledger</code> , <code>DeliveryTag</code> and AMQP headers — in the interface. Every consumer, generated mock and test must import <code>amqp091-go</code> , and the port cannot be satisfied by SQS, NATS or an in-memory fake.
29	TestRepo_MinIO_PresignedUploadHasASizeCeiling	infra/minio/storage_repo.go	No upload size limit anywhere. <code>PresignedPutObject</code> signs method, bucket, key and expiry only — nothing else. <code>StorageRepository.Create</code> has no size parameter to express one, the temp bucket has no quota or content-length-range policy, and <code>middleware.MaxBody</code> cannot help because bytes go browser → MinIO directly. A holder of an upload URL can fill the disk.

4 Corrections to the brief

Three rows in the supplied weak-points table were stale against the current source. Tests were still written to the correct contract; all three now pass and guard against regression.

#	CLAIM IN THE BRIEF	ACTUAL STATE	OUTCOME
1	"gifRepo.Update body is <code>_ = db; return nil</code> "	It runs a real <code>UPDATE ... RETURNING</code> at <code>infra/postgres/gif_repo.go:104</code> . <code>SaveRecent</code> at line 123 is still a no-op — that half of the claim holds and is covered by a gated integration test.	PASS
2	"Update handler never decodes the request body"	It decodes into a <code>media.GifUpdateRequest</code> at <code>handlers/media/update.go:32</code> . A gomock argument matcher now pins the decoded fields, including that an absent field stays <code>nil</code> so the repo's <code>COALESCE</code> works.	PASS
3	"share.Create never checks <code>sharedBy</code> is the owner"	It does — <code>app/share/create.go:30</code> compares <code>GetOwner</code> against <code>sharedBy</code> and returns <code>ErrGifOwnerMismatch</code> . The test is retained so the check cannot be quietly removed.	PASS

A bug the brief did not list — and it should crash in production

`infra/postgres/gif_repo.go:74` appends the row lock with no separating space:

```
if forUpdate == true {
    query += "for update"           // → "... where key = $1for update"
}
```

Postgres rejects that as a syntax error. Every write path runs `Update` → `tnx.WithRC` → `GetByKey(key, true)`, so the locked read inside the update transaction can never succeed. `TestRepo_GifGetByKey_ForUpdateTakesARowLock` proves it, but needs a live Postgres to run.

Other defects found beyond the brief

- **Share emails are never delivered** — no `case "share"` in the email worker (finding 25).
- **Stream has no authorization at all** (finding 2) — arguably the most exploitable issue in the report.
- **Signup's `Location` header is a string literal** (finding 21).
- **Rate limiter panics on four malformed replies** (finding 26).
- **Account enumeration on forgot-password** (finding 4).
- **The architecture leak is in `port/queue`, not `domain/media`** — the domain layer is clean; `domain/media/storage.go` does not import the MinIO SDK (finding 28).

5 Coverage of the brief's weak-points table

AREA FROM THE BRIEF	RESULT	NOTE
GIF update persistence	NEEDS PG	Not a no-op; round-trip test written, gated on Postgres.
Save-to-recents persistence	NEEDS PG	Confirmed no-op by reading. Integration test asserts the row actually changes.
Update handler body parsing	PASS	Handler decodes correctly; matcher pins the fields.

AREA FROM THE BRIEF	RESULT	NOTE
Share ownership check	PASS	Check present at <code>create.go:30</code> .
Auth blocklist fail-open	FAIL	Confirmed — finding 1, three outage modes.
Cache miss vs. backend error	FAIL	Confirmed, inverted: a miss produces 500 — finding 14.
Reset token hashing	FAIL	Confirmed — finding 5.
Convert cross-field validation	FAIL	Confirmed — finding 18. <code>start=5,end=5</code> also accepted.
DLQ vs. retry	FAIL	Confirmed across all three workers — findings 22–24.
Rate limiter burst bug	NEEDS REDIS	Test drains a fresh bucket and asserts <code>capacity</code> , not <code>rate</code> . Also written: the hardcoded <code>PEXPIRE 6000</code> evicts slow-refill buckets, letting a caller reset their own limit.
QoS prefetch	NEEDS RABBITMQ	Asserts prefetch equals the worker pool size end-to-end, rather than merely that some QoS was set.
Presigned upload size	FAIL	Documented gap, asserted statically so it runs without MinIO — finding 29.
Domain/infra leak	FAIL	Domain is clean; the real leak is <code>amqp</code> in <code>port/queue</code> — finding 28. Guard is a transitive import-graph walk.
Quota enforcement	FAIL	Confirmed — findings 11–13, including the concurrency case.

6 A note on the clean sheet

All 15 tests in `internal/transport/http/handlers/user` pass. Per the brief's own hard rule I treated that as a smell and went back over it: the user handlers genuinely do check the auth context, validate with `eqfield`, require `If-Match` on profile updates, and take the user id from the token subject rather than the request body — which the test verifies by sending a hostile `"user_id"` in the payload. Reporting no defects there rather than manufacturing one.

The concurrency tests are likewise clean under `-race`: 32 requests racing a logout through the blocklist, 16 goroutines creating a duplicate share pair, and 12 concurrent renames through the `FOR UPDATE` path all behave consistently. **0 data races across the whole suite.**

7 Running it

```
# unit, handler, worker, concurrency and architecture layers – all run now
.devtools\go.cmd test -race ./...

# regenerate mocks after an interface change
.devtools\go.cmd generate ./...

# the 14 gated integration tests (needs Docker)
docker compose up -d
.devtools\go.cmd test ./tests/integration/... -run Repo -v
```

Nothing was patched. This first pass surfaces defects through tests only, as specified — the 29 failures are the deliverable, not a regression.

